

Orchestration Fundamentals for Agentic Development

Planning and running multi-agent development work with Claude Code



About Me:

- Instructor
- Living in San Francisco
- 9+ years specializing in ML/AI education and curriculum design
- I've used LLMs to prototype an adaptive coding learning platform





About You!

- Name
- Role
- What brought you here, and what are you most excited to learn about?



TODAY

Agenda

0:00 Kickoff & environment check

0:20 Module 1 — Decomposing Complex Features (lecture + lab)

1:20 *Break*

1:30 Module 2 — Model Selection Strategy (lecture + lab)

2:15 *Break*

2:25 Module 3 — Orchestrating Multiple Agent Sessions (lecture + lab)

3:45 Wrap-up & Q&A

Learning Objectives

1

Break down complex features into units of work suited for agent execution

2

Select an appropriate model for a task based on its complexity, risk, and cost

3

Run and coordinate multiple Claude Code sessions on related subtasks

Prerequisites

This session is hands-on from the first hour. Here is what it assumes you bring.

EXPERIENCE

1

An AI coding assistant

You have used Claude Code or a similar tool for real development tasks.

2

Git branching and merging

You can create a branch, commit to it, and merge it back.

3

The terminal

You are comfortable navigating directories and running commands.

SETUP

Claude Code CLI

installed and authenticated

Node.js

required by the Claude Code CLI

Git

with access to push branches to the course repo

Python 3.10+

the practice app is a small Flask project

A terminal and a code editor

VS Code or any editor you like

Network access

outbound calls to Anthropic's API not blocked by a proxy

BEFORE WE START

Environment Check

Run these now. Flag a failure before Module 1 starts, not during the lab.

```
claude --version
```

```
python3 --version
```

```
git --version
```

Also confirm: Claude Code is authenticated, and you have access to the taskboard repo.

WHY ORCHESTRATE?

From One Assistant to a Team

Same feature, two ways to run it.

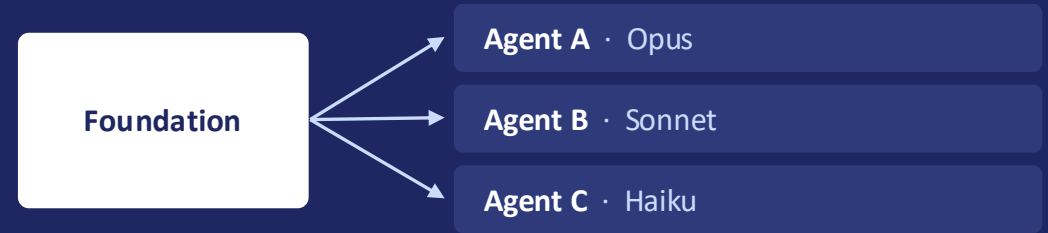
ONE SESSION



one after another, one context, one model

- Each step waits for the one before it.
- One context fills up with every file and decision.
- One model for everything, trivial or risky.
- One big diff to review at the end.

ORCHESTRATED



first the foundation, then parallel units, a model for each

- Independent units run at the same time.
- Each agent sees only the files for its unit.
- Each unit runs on a model that fits its risk.
- Small diffs, each checked against its own criteria.

WHY ORCHESTRATE?

What You Gain, and What It Costs

Orchestration is a trade. Worth it when the work splits cleanly; skip it when the task is small or every piece touches the same files.

WHAT YOU GAIN

Speed Independent units run in parallel.

Focus Each agent's context holds only its unit.

Fit The right model and effort for each task.

WHAT IT COSTS

Coordination Someone plans, sequences, and merges.

Conflicts Parallel edits to shared files collide.

Spend More sessions means more tokens.

TODAY'S MODULES EACH ANSWER A COST

MODULE 1

Decomposition: clear specs, no shared files

MODULE 2

Model selection: spend matched to risk

MODULE 3

Orchestration: worktrees and merge discipline

MODULE 1

Decomposing Complex Features for Multi-Agent Execution

Lecture 22 min • Lab 35 min

What Makes a Task “Agent-Ready”?

1

Description (with a carefully bounded scope)

A clear task, on an identifiable set of files — one coherent thing, not “build the backend.”

2

Acceptance criteria

Checkable without reading the whole diff.

3

Named dependencies

What must exist first, and what this blocks.

4

A verification step

A test, a curl, a page you can look at.

Description (with a carefully bounded scope)

A coherent change to a small set of files you can name before the agent starts.

HOW TO WRITE IT

- 1 List the files it will touch, and the ones it must not.
- 2 State the outcome in one sentence. If you need the word “and,” split it.
- 3 Point at an existing pattern to copy: “follow how orders.py does it.”
- 4 Size it so the diff can be reviewed in one sitting.

EXAMPLES

TOO BROAD

“Add search to the store.”

BOUNDED

Add GET /products/search?q= in api/products.py: case-insensitive match on product name. No UI changes.

TOO BROAD

“Improve login security.”

BOUNDED

Lock an account for 15 minutes after 5 failed logins. Changes limited to auth/login.py and the users table.

TOO BROAD

“Clean up the billing code.”

BOUNDED

Move the tax math in invoice.py into calculate_tax(subtotal, region). No change in behavior.

Acceptance Criteria

A pass/fail checklist the agent can check itself against, and you can confirm without reading every line.

HOW TO WRITE IT

- 1 Write 3–5 criteria. Each one is clearly true or false.
- 2 Describe observable behavior: return values, status codes, function names.
- 3 Always include “existing tests still pass, unmodified.”
- 4 Cut adjectives. “Clean,” “robust,” and “fast” can’t be checked.

EXAMPLES

VAGUE

“Search works well.”

CHECKABLE

q=lamp returns every product with “lamp” in its name, in any case. An empty q returns 400.

VAGUE

“Handle lockouts properly.”

CHECKABLE

The 6th failed attempt within 15 minutes returns 423. A successful login resets the counter to 0.

VAGUE

“The dashboard is fast.”

CHECKABLE

The dashboard query finishes in under 200 ms against the 10,000-row seed dataset.

Named Dependencies

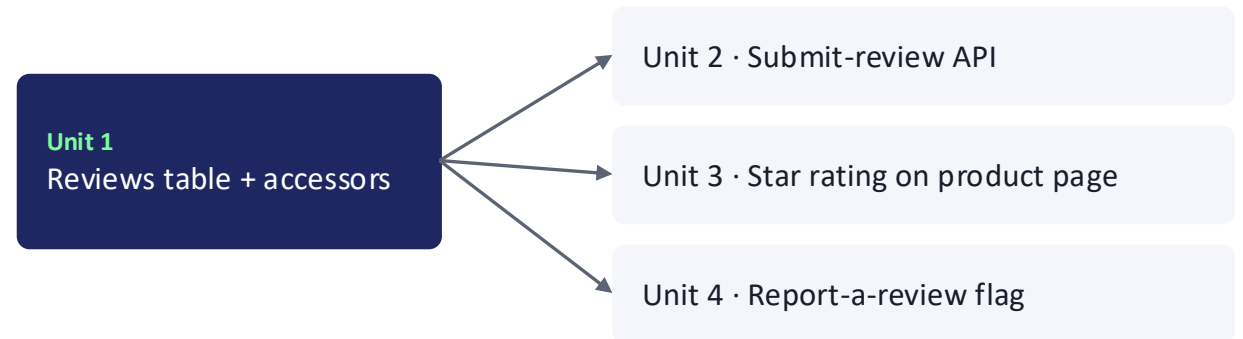
State what must exist before this unit starts, and which units are waiting on it.

HOW TO WRITE IT

- 1 “Depends on:” the exact tables, functions, or endpoints it needs.
- 2 “Blocks:” the downstream units that assume this one’s output.
- 3 Pin down the contract (names and signatures) so others can start against it.
- 4 No shared files and no dependency between two units means they can run in parallel.

EXAMPLES

Feature: product reviews



Units 2–4 share no files and don’t depend on each other, so they can run side by side.

Unit 3 · Star rating on product page

Depends on: Unit 1, specifically `get_average_rating(product_id) → float`

Blocks: nothing

Shares files with: none

A Verification Step

How you, not the agent, confirm it's done. One concrete check that takes under two minutes.

HOW TO WRITE IT

- 1 Write the exact command, URL, or click path.
- 2 Put the expected result right next to it.
- 3 Pick the cheapest check that would still catch a wrong answer.
- 4 The agent saying “done” is not verification. You run the check.

EXAMPLES

TEST

```
pytest tests/test_tax.py -v
```

Expect: 6 passed, including 3 new region cases

CURL

```
curl -i  
"localhost:8000/products/search?q="
```

Expect: HTTP 400 Bad Request

PAGE

```
Open /products/42
```

Expect: 4.5 stars shown; a product with no reviews says “No reviews yet”

LOG

```
Fail login 5× as  
test@example.com
```

Expect: Log line: account locked: test@example.com

Good Unit of Work vs. Bad

BAD

“Add workspaces to the app.”

- Unbounded scope — touches schema, routes, templates, permissions at once
- No acceptance criteria
- No stated dependencies
- No way to verify partial progress

GOOD

Workspace & membership schema... with named accessor functions, acceptance criteria, and a stated “blocks” list.

- Existing tests still pass unmodified
- New accessor functions named explicitly
- A new test file covers the new behavior
- Depends on nothing; blocks three other units

Unit 1: From Feature Request to Description

Matching highlights show where each part of the description comes from.

THE FEATURE REQUEST (UNIT 1 PARTS)

1. **Workspaces.** A workspace is a named container for tasks (e.g. "Engineering," "Marketing"). Every task belongs to exactly one workspace. Every user can belong to multiple workspaces.
2. **Roles.** Within a workspace, a user is either an **admin** (can create and delete tasks, invite/remove members, change any task) or a **member** (can create tasks and edit tasks assigned to them, but cannot delete tasks or manage membership).

The data model has a real dependency chain: nothing else can be built until workspaces and workspace membership exist in the schema. That's deliberate — it's the sequencing example for Module 1.

UNIT 1 DESCRIPTION

Add the data model that everything else depends on. In `app/db.py`, add a `workspace` table (`id`, `name`) and a `workspace_member` join table (`workspace_id`, `user_id`, `role`, where `role` is `admin` or `member`), following the existing style — plain `sqlite3`, schema in the `SCHEMA` string, accessor functions below it, no ORM. Add a nullable `workspace_id` column to the existing `task` table so current rows don't break. `taskboard.db` already exists, so `init_db()` must also add the column to existing databases: if `task` has no `workspace_id` column, run `ALTER TABLE task ADD COLUMN workspace_id INTEGER`. Add accessor functions: `create_workspace(name)`, `add_member(workspace_id, user_id, role)`, `list_workspaces_for_user(user_id)`, `get_member_role(workspace_id, user_id)`.

Not highlighted: not in the request. It comes from reading the codebase.

Unit 1: From Description to Acceptance Criteria

For each piece of the description, ask how you would check it without reading the diff.

THE DESCRIPTION SAYS	ASK	ACCEPTANCE CRITERION
<i>add a workspace table... a workspace_member join table... a nullable workspace_id column</i>	→ Is everything I described actually in the schema?	→ SCHEMA defines workspace and workspace_member, and task gains a nullable workspace_id column.
<i>so current rows don't break</i>	→ What works today that must keep working?	→ All four existing tests in tests/test_tasks.py still pass unmodified.
<i>Add accessor functions: create_workspace... get_member_role</i>	→ Does each named function exist, and what is its trickiest case?	→ create_workspace, add_member, list_workspaces_for_user, and get_member_role all exist and behave as named (e.g. get_member_role returns None for a non-member rather than raising).
<i>list_workspaces_for_user... role, where role is admin or member</i>	→ Which behaviors need a test to prove them?	→ A new tests/test_workspaces.py covers: creating a workspace, adding a member with a role, listing workspaces for a user who belongs to several, and confirming a non-member gets no role back.
<i>In app/db.py... following the existing style</i>	→ What should this unit not touch?	→ No changes to app/routes.py or templates in this unit — scope stays in app/db.py and its test file.
<i>taskboard.db already exists, so init_db() must also add the column</i>	→ What already exists that this has to work with?	→ Against the existing taskboard.db (without deleting it), existing tasks are kept, task has a workspace_id column, and python seed_demo.py runs cleanly and reports the Engineering and Marketing workspaces.

The Schema Unit: Description and Criteria

Unit 1 of today's feature request, written out step by step. This one is given; you'll write Units 2–4.

1 Description

Touches: `app/db.py`, plus a new `tests/test_workspaces.py`

Adds: `workspace (id, name)` and `workspace_member (workspace_id, user_id, role), role 'admin' or 'member'`

Alters: `task` gets a nullable `workspace_id`, so existing rows don't break; migrated into the existing `taskboard.db`

Accessors: `create_workspace`, `add_member`, `list_workspaces_for_user`, `get_member_role`

Follows: plain `sqlite3`, schema in the `SCHEMA` string, accessors below it. No ORM.

2 Acceptance criteria

- ☐ `SCHEMA` defines `workspace` and `workspace_member`; `task` gains a nullable `workspace_id`
- ☐ All 4 tests in `tests/test_tasks.py` still pass, unmodified
- ☐ The four accessors behave as named: `get_member_role` returns `None` for a non-member instead of raising
- ☐ `tests/test_workspaces.py` covers `create`, `add member` with a role, `list` for a user in several workspaces, and a non-member getting no role
- ☐ No changes to `app/routes.py` or templates
- ☐ Against the existing `taskboard.db`, `seed_demo.py` runs cleanly and reports both workspaces

The Schema Unit: Dependencies and Verification

Unit 1 of today's feature request, written out step by step. This one is given; you'll write Units 2–4.

3 Named dependencies

Depends on: nothing. This is the foundation, so it runs first, on its own.

Blocks: Units 2, 3, and 4. None of them can be written against real tables until this lands.

Contract: the table and column names (fixed by `seed_demo.py`) and the four accessors. Unit 2's permission checks call `get_member_role`.

Unit 1 · Schema



Units 2, 3, and 4

4 Verification step

```
python -m unittest discover -s tests -v
```

Expect: the 4 original tests plus the new workspace tests, all passing.

```
python seed_demo.py
```

Expect: reports both workspaces, using the existing database.

```
python -c "import sqlite3; print([r[1] for r in
sqlite3.connect('taskboard.db').execute('PRAGMA
table_info(task)'])]"
```

Expect: the column list ends with `workspace_id`.

MODULE 2

Model Selection Strategy for Agentic Development

Lecture 15 min • Lab 30 min

Match the Model to the Task

Model	Best for	Profile
Haiku	Small, well-specifieded, low-risk	<i>Fast, cheap</i>
Sonnet	Default for day-to-day implementation	<i>Balanced</i>
Opus	Architecture decisions, foundational/shared code, higher risk	<i>Deeper reasoning</i>
Fable	Long, ambiguous, investigative work	<i>Most capable, not default</i>

Effort level (low / medium / high / xhigh / max) is a separate dial from model choice.

Choosing Effort, and When to Step Up

Effort is how much the model thinks and checks before it acts. Start at the lowest level that fits the risk, and raise it on evidence.

PICK THE EFFORT LEVEL

low

One obvious change, nothing to decide.
e.g. Rename a field, add a static route

medium

Clear spec, some logic, tests to write.
e.g. Add validation to an existing endpoint

high

Design calls, or existing behavior that must not break.
e.g. Change a shared data model

xhigh

Ambiguous. It should explore before committing.
e.g. Recommend an approach with tradeoffs

max

Rarely. The hardest problems, after high or xhigh fell short.
e.g. A subtle bug nothing else has cracked

IT STRUGGLED. NOW WHAT?

RAISE EFFORT WHEN IT WAS CARELESS

- Missed an edge case or skipped tests
- Didn't check its own work against the criteria
- Gave a shallow answer to a question that needed digging

SWITCH TO A STRONGER MODEL WHEN IT DIDN'T UNDERSTAND

- Misread the task, or solved a different problem
- Repeats the same mistake after you correct it
- The change is foundational and blocks other work

Rule of thumb: more effort fixes carelessness; a stronger model fixes misunderstanding. Easy work coming back fast and right? Step down.

```
claude --effort high /model opus
```

Switching Models Mid-Session

```
/model          # open picker  
  
/model opus     # switch directly  
  
/model sonnet  
  
s              # (in picker) apply to this session only
```

Escalate mid-task when a Sonnet session is stuck or the stakes turn out higher than expected. Downgrade when a task is proving simpler than planned.

MODULE 3

Orchestrating Multiple Agent Sessions

Lecture 20 min • Lab 60 min

Git Worktrees vs. Alternatives

ALTERNATIVES

Separate clones, or one directory switching branches

- Full clone per task — duplicates git history and installed dependencies every time
- Branch-switching in one directory — sequential only, and clobbers uncommitted work
- Multiple sessions, one shared directory — uncommitted edits can overwrite each other
- No shared history — new commits need a push/pull before another checkout can see them

WORKTREES

One shared .git history — a separate working directory and branch per session.

- One shared .git object store — no duplicated history, no clone per task
- Separate working directory per branch — truly parallel, nothing to clobber
- New commits visible instantly across worktrees — no push/pull needed
- Created and torn down in seconds — git worktree add / remove

Git Worktrees: Parallel Sessions

1

Land dependencies first

Foundational/sequential work (e.g. a schema change) goes on main, alone.

2

Create one worktree per independent unit

`git clone --worktree <name> —` separate directory, separate branch.

3

Verify inside each worktree

Run tests before merging anything back.

4

Merge one at a time

Verify between each merge, not after all of them.

Least Privilege & Merge Discipline

```
tools: Read, Edit, Write, Bash  
model: sonnet
```

Omitting the tools field grants full access — not zero. Research-only agents should never get Write or Edit.

Merge discipline

- Merge units one at a time
- Run tests after each merge, not after all of them
- A conflict may mean the units weren't as independent as planned
- Manually exercise the feature end to end before calling it done

Recap & Q&A

Decompose before you delegate — scope, criteria, dependencies, verification

Match model + effort to complexity, risk, and cost

Worktrees + merge discipline turn agents into a team, not a queue

Keep the quick-reference cheat sheet — model table, worktree commands, tool-scoping snippet.

Questions?